

Kubernetes Migration Plan

ShopFast — E-Commerce Microservices Platform

Spring Boot · 16 Microservices · Eureka · Keycloak · Kafka · PostgreSQL · Redis · Elasticsearch

Prepared: August 2026 | Author: Platform Engineering

1. Executive Summary

ShopFast is a production-grade e-commerce platform built as a set of Spring Boot microservices (16+ services) with service discovery (Eureka), identity management (Keycloak), an event backbone (Kafka), relational and in-memory data stores (PostgreSQL, Redis), full-text search (Elasticsearch), and centralized logging (ELK). Today the system is orchestrated with Docker Compose. This document defines a step-by-step plan to migrate the platform to Kubernetes, improving scalability, resilience, observability, and deployment automation.

The migration is incremental and low-risk: infrastructure services are deployed first via Helm, followed by stateless microservices using a reusable Kustomize template, then ingress, observability, and finally GitOps-based CI/CD.

2. Current Architecture (As-Is)

Layer	Components
Microservices (16)	eureka-server, api-gateway, auth, user, product, category, inventory, cart, coupon, order, payment, review, notification, admin, elastic-service, frontend
Discovery	Netflix Eureka
Identity	Keycloak 26.0
Messaging	Kafka 7.6.1 + Zookeeper
Datastores	PostgreSQL 15, Redis 7, Elasticsearch 8.19
Logging	Filebeat / Logstash / Elasticsearch / Kibana (ELK)
Edge / Proxy	Nginx (shopfast.live, shopfast_test.live)
Orchestration	Docker Compose
Build	Per-service multi-stage Maven Dockerfiles

Phase 0 — Prerequisites & Environment

- Stand up a local cluster: Docker Desktop Kubernetes or `minikube start --cpus=4 --memory=8192 --driver=docker`.
- Install tooling: **kubectl**, **helm**, **kustomize** (bundled), **scaffold** (dev loop), **k9s** (optional UI).
- Choose a container registry: local minikube registry for dev, Docker Hub / GHCR for real deploys. Define namespace e.g. `shopfast/<service>:<tag>`.

Phase 1 — Container Image Prep

- Build **common-lib** once and publish it so each service image no longer rebuilds it.
- Tag images by **git SHA** (not latest) to enable rollbacks.
- Shrink images: switch the run stage from JDK to `eclipse-temurin:21-jre`.
- Automate builds with `scripts/build-images.sh` or a `scaffold.yaml`.

Phase 2 — Namespace & Config Strategy

- Create `k8s/overlays/` (base + dev/prod) using **Kustomize**.
- Namespace `shopfast` (dev) and `shopfast-prod`.
- Externalize config: non-secret values → **ConfigMaps**; passwords, Keycloak secrets, JWT keys → **Secrets**.
- Keep Eureka for discovery; point services to `eureka-server.shopfast.svc.cluster.local` (optionally later replace with K8s Service DNS).
- Inject via `envFrom: configMapRef / secretRef`.

Phase 3 — Stateful Backing Services (Deploy First)

- **PostgreSQL 15** — Helm `bitnami/postgresql` + PVC 10Gi + Secret.
- **Redis 7** — Helm `bitnami/redis` (or `redis-ha`).
- **Kafka + Zookeeper** — Helm `bitnami/kafka` or the Strimzi Operator; create topics via init job or KafkaTopic CRDs.
- **Elasticsearch 8** — Helm `elastic/elasticsearch` + Kibana. Reuse existing Filebeat (DaemonSet) and Logstash (Deployment) reading pod stdout.
- **Keycloak 26** — Helm `bitnami/keycloak` / `codecentric/keycloak`, import realm.

Phase 4 — Stateless Microservices

- For each of the 16 services: **Deployment** with 2 replicas, resource requests/limits (e.g. 512Mi/1Gi).
- Add **readiness**, **liveness**, and **startup** probes on Actuator health endpoints for safe rollouts.
- Create a **Service** (ClusterIP) per service.
- Add **HPA** on CPU > 70% for busy services (gateway, order, product).
- Generate uniformly from a Kustomize base + per-service patches or a templated Helm chart.

Phase 5 — Ingress & Routing

- Deploy **ingress-nginx**; map existing `Nginx/shopfast.live` path rules to an Ingress resource.
- Build & deploy the **frontend** image; expose it and api-gateway via the same Ingress hosts.
- Enable **TLS** with cert-manager + Let's Encrypt (self-signed for dev) for `shopfast.live` / `shopfast_test.live`.

- Route `/api/*` to `api-gateway`; let the gateway fan out internally.

Phase 6 — Observability (ELK already present)

- **Logging**: Filebeat DaemonSet → Logstash → Elasticsearch → Kibana (reuse existing configs).
- **Metrics**: Prometheus + Grafana via `kube-prometheus-stack`; scrape Actuator `/actuator/prometheus`.
- **Tracing** (optional): Spring Cloud Sleuth → Zipkin/Jaeger.
- Build per-service Grafana dashboards and node-health views.

Phase 7 — Resilience & Security

- Apply **Resilience4j** circuit breakers on the `api-gateway`.
- Add **NetworkPolicies**: deny-by-default; only `api-gateway` reachable from ingress.
- Secrets: seal with **SealedSecrets** or **External Secrets Operator** (never commit raw Secrets).
- Harden pods: non-root `securityContext`; set namespace **resource quotas**.

Phase 8 — CI/CD

- Extend `.github/workflows/docker-image.yml`: test → build → push → apply overlay.
- Adopt **GitOps** with **Argo CD** syncing `k8s/overlays/prod` from git for easy rollbacks.
- Add **Trivy** image scanning in CI before push.

Phase 9 — Local Dev Workflow

- Use `scaffold dev` for hot-reload on a single service.
- Use `kubectl port-forward` to debug DBs/Kafka/Keycloak.
- One-command bring-up: `kubectl apply -k k8s/overlays/dev`.

3. Suggested Repository Layout

```
k8s/  
base/ <- Deployments, Services, ConfigMaps (shared)  
overlays/  
dev/  
prod/  
infra/ <- Helm releases / operators (pg, redis, kafka, es, keycloak)  
ingress/ <- Ingress + TLS + cert-manager  
scripts/  
build-images.sh  
deploy.sh
```

4. Migration Order (Low Risk → High Risk)

- 1. Namespace + ConfigMaps/Secrets scaffolding
- 2. Infra: PostgreSQL, Redis, Kafka, Elasticsearch, Keycloak
- 3. Eureka + api-gateway
- 4. Leaf services: product, category, user, auth
- 5. Order / cart / payment / notification flow
- 6. Frontend + Ingress + TLS
- 7. Observability + HPA + NetworkPolicies
- 8. CI/CD + Argo CD + prod overlay

5. Key Risks & Mitigations

Risk	Mitigation
Eureka vs K8s DNS overlap	Keep Eureka initially; migrate to K8s Service discovery per service once stable.
Stateful data loss	Use PersistentVolumeClaims with backups; test restore before cutover.
Image bloat / slow boot	JRE base, readiness/startup probes, git-SHA tags, resource limits.
Secret leakage	SealedSecrets / External Secrets; scan images with Trivy.
Config drift between envs	Kustomize overlays + Argo CD GitOps single source of truth.

Generated as part of the ShopFast Kubernetes migration planning. Next step: implement Phase 2–4 (Kustomize base + reusable service template + infra Helm installs) to stand up a working vertical slice.